

Les fondations du programmation Python

Abdallah Houmadi SOIRFANE

31 juillet 2026

1 Introduction

La programmation, c'est quoi ?

Si vous demandez à un ingénieur, il vous parlera de boucles, de fonctions, de gestion de mémoire. Si vous demandez à un prof, il vous parlera de logique et d'algorithmique.

Mais si vous demandez à votre pote qui a déjà essayé, il vous répondra : *"C'est un truc qui te rend fou pendant trois heures, et quand ça marche enfin, tu te sens plus fort que Dieu."*

La première réaction qu'on a quand on ouvre un terminal, c'est : *"Mais qu'est-ce que je dois taper ? Pourquoi ça plante ? Et surtout, est-ce que l'ordinateur me prend pour un con ?"*

Pour bien comprendre ce qu'est Python, faisons un petit pas en arrière. Pas un Big Bang, juste un petit pas.

Un ordinateur, c'est un cerveau hyper puissant, mais complètement débile. Il est capable de faire des milliards de calculs par seconde. Mais si vous ne lui dites pas *exactement* quoi faire, il ne fait rien. Il attend. Il est bête, mais il est rapide.

Python, c'est le langage qu'on utilise pour parler à ce cerveau débile. C'est une langue. Une langue avec des mots ("print", "if", "for") et une grammaire ultra stricte (les deux-points, les indentations, les parenthèses). Si vous respectez la grammaire, le cerveau exécute votre ordre. Si vous faites une faute, il vous insulte avec un message d'erreur rouge. C'est une relation compliquée. Je l'avoue.

Dans ce cours, on va apprendre à parler cette langue. Pas pour devenir des pros, mais pour être capables de lui dire *"Tiens, regarde cette image, dis-moi ce que c'est"*. Et ça, c'est déjà de la magie.

2 Variables et Types de Données

2.1 Le problème du jour : la mémoire de poisson rouge

Imaginez que vous arrivez dans un nouveau lycée, disons le lycée de Bambao. Vous ne connaissez personne. On vous présente votre meilleur pote de la journée : il s'appelle « Youssouf » (oui, c'est son vrai nom), il a 17 ans, il mesure 1,82 m, et il est absolument convaincu d'être le prochain Mbappé.

Vous vous dites : « Je vais retenir tout ça, c'est facile. »

Deux heures plus tard, vous croisez un grand gars dans le couloir. Vous lui dites : « Salut... euh... Youssef ? Ahmed ? » Il vous regarde avec un air de chien battu. Raté.

Le problème, c'est que votre cerveau est une passoire. Il oublie. Il mélange. Il invente.

Maintenant, remplacez votre cerveau par un ordinateur. C'est pareil, mais en pire. Un ordinateur, il ne retient absolument rien. Il est aussi vide qu'un frigo après une soirée pizza. Chaque fois que vous le redémarrez, il fait une amnésie totale.

Alors, comment fait-on pour qu'un ordinateur se souvienne de « Youssef » ?

On va lui donner des **étiquettes** et des **boîtes**.

2.2 Vous avez dit « Variables » ?

Une **variable**, c'est exactement ça : une boîte rangée dans la mémoire de l'ordinateur, avec une étiquette collée dessus.

Vous écrivez un nom sur l'étiquette (le **nom de la variable**), et vous mettez un truc dedans (la **valeur**). Quand vous voulez ressortir le truc, vous appelez la boîte par son nom.

Python, lui, a une particularité : il est un peu fainéant. Il ne veut pas qu'on lui dise en avance ce qu'il va mettre dans la boîte. Il préfère regarder le contenu et décider tout seul. C'est ce qu'on appelle le **typage dynamique**. En gros, il juge le livre par sa couverture.

Regardez :

```
1 message = "Bonjour"
2 nombre = 25
3
```

Ici, Python regarde "Bonjour" avec ses guillemets, il se dit « Ah, c'est du texte ! » et il range ça dans la boîte message. Puis il regarde 25, il voit que c'est un chiffre sans virgule, et il le met dans nombre. Il n'a pas eu besoin de préciser. C'est pratique, non ?

Ce que vous devez retenir :

- ① Une variable, c'est une boîte avec une étiquette. L'étiquette, c'est le nom. Le contenu, c'est la valeur.
- ② Vous ne pouvez pas avoir une boîte vide. En programmation, une variable non initialisée, ça n'existe pas. C'est comme un yaourt sans yaourt dedans : ce n'est plus un yaourt.

2.3 Les différents types de boîtes (ou comment ne pas mettre une chaussette dans une boîte à chaussures)

Python est malin, mais il n'est pas devin. Si vous lui mettez du texte dans une boîte, il ne pourra pas faire de calculs avec. Et si vous lui mettez un nombre, il ne pourra pas le transformer en poème. Il faut donc connaître les sortes de boîtes qui existent.

Heureusement, il n'y en a que quatre à retenir pour commencer :

- ❶ **Les chaînes de caractères (str)** : c'est du texte. N'importe quel texte. Et pour que Python comprenne que c'est du texte, on le met entre guillemets (simples ou doubles, il s'en fiche). C'est comme une étiquette qui dit : « Attention, contenu fragile, à manipuler comme du texte. »
- ❷ **Les entiers (int)** : des nombres ronds. Des vrais. Sans virgule. 3, -12, 2026. C'est le genre de nombre qu'on utilise pour compter des moutons.
- ❸ **Les flottants (float)** : des nombres à virgule. En informatique, on dit « flottant » parce que la virgule « flotte » (en vrai, c'est un point, mais on va pas chipoter). 3.14, -0.5, 1.82. C'est pour les mesures, les prix, les notes de maths.

- ④ **Les booléens (bool)** : c'est soit **True** (vrai), soit **False** (faux). C'est la réponse à toutes les questions que vous vous posez. Est-ce que Youssouf est le prochain Mbappé? False. Est-ce que vous avez compris les variables? True (on croise les doigts).

Mais comment savoir ce qu'il y a dans une boîte sans l'ouvrir? Python nous donne une petite lampe torche : la fonction `type()`.

```
1     nom = "Soirfane"           # str (String, Chaîne de caractères)
2     age = 19                   # int (Entier)
3     taille = 1.70              # float (Reel)
4     etudiant = True            # bool (Booleen)
5
6     print(type(age))           # Affiche <class 'int'>
7
```

Et là, vous me dites : « Mais c'est quoi ce truc `print`? C'est quoi cette affaire de `type()`? »

Eh bien, pour l'instant, retenez juste que `print()` est la voix de l'ordinateur. C'est lui qui crie ce qu'il a dans le ventre. Et `type()`, c'est un détective qui va fouiller dans votre boîte pour vous dire ce qu'il y a dedans. Lancez-le, il vous rendra service.

2.4 Les Post-it : les commentaires

Vous avez remarqué, dans les codes ci-dessus, il y a des lignes qui commencent par un symbole `#`. C'est ce qu'on appelle un **commentaire**.

Les commentaires, c'est le petit mot doux que vous laissez dans votre code pour expliquer à vous-même (ou à votre pote qui va relire votre travail dans trois mois) ce que vous avez fait.

L'ordinateur, lui, il s'en fiche. Il ne les lit pas. Il les ignore. C'est comme un Post-it collé sur votre frigo : il est là, tout le monde le voit, mais le frigo continue de faire son boulot sans s'en soucier.

```
1     # Je declare une variable de type <str> et je l'affecte a la valeur <
    Soirfane>
2     nom = "Soirfane"
3
```

Pourquoi écrire des commentaires? Parce qu'un jour, vous allez relire votre code et vous dire : « Mais pourquoi j'ai mis 42 ici?! » Et là, si vous avez un commentaire, vous vous souviendrez que 42 est la réponse à la vie, l'univers et le reste. Sinon, vous passerez deux heures à chercher.

Alors, prenez l'habitude dès maintenant. Un petit `#` par ci, un petit `#` par là. Votre futur vous vous remerciera.

3 Opérations de base et Structures de contrôle

3.1 Opérations de base

3.1.1 Le problème du jour : le marché de Volo-Volo

Imaginez que vous êtes au marché de Volo-Volo. Vous avez 10 000 francs comoriens en poche. Vous voulez acheter 3 kg de bananes à 1 500 francs le kilo, et 2 kg de mangues à 2 000 francs le kilo.

Vous sortez votre calculatrice (ou votre cerveau, si vous êtes courageux) et vous faites le calcul :

$$3 \times 1500 + 2 \times 2000 = 4500 + 4000 = 8500$$

Il vous reste 1 500 francs. Vous pouvez encore acheter un soda. Vous êtes content.

Maintenant, imaginez que vous deviez faire ça 10 000 fois par jour. Pour un ordinateur, c'est son métier. Il est né pour ça. Il fait des calculs plus vite que vous ne dites « Mbappé ». Mais il faut lui donner les bonnes instructions.

C'est là qu'interviennent les **opérations arithmétiques**.

3.1.2 Les opérations arithmétiques : le calcul mental de l'ordinateur

Python sait faire des calculs. Tous les calculs. Il est une machine à calculer surpuissante déguisée en langage de programmation. Les opérations de base, vous les connaissez déjà :

- ❶ **Addition** : + (pour ajouter des bananes)
- ❷ **Soustraction** : - (pour enlever le prix du soda)
- ❸ **Multiplication** : * (pour savoir combien coûtent 3 kilos)
- ❹ **Division réelle** : / (pour partager équitablement)
- ❺ **Division entière** : // (pour savoir combien de parts entières on peut faire)
- ❻ **Puissance** : ** (pour mettre en exposant)

```
1 # Operation arithmetique
2 x = 1
3 y = 3.6
4 print(x + y)    # L'addition de x et y
5 print(x - y)    # La soustraction de x et y
6 print(x * y)    # La multiplication de x par y
7 print(x / y)    # La division reelle de x par y
8 print(x // y)   # La division entiere de x par y
9 print(x ** y)   # x exposant y
```

3.1.3 Opérations de comparaison : le juge de la vie

Vous avez déjà fait des comparaisons dans votre vie. Par exemple :

- « Est-ce que Youssouf est plus grand que Soirfane ? »
- « Est-ce que j'ai assez d'argent pour acheter un soda ? »
- « Est-ce que mon équipe de foot a marqué plus de buts que l'autre ? »

Le résultat, c'est toujours **vrai** ou **faux**. Il n'y a pas de « peut-être » en programmation. C'est binaire. C'est tranché. C'est la vie.

En Python, on compare avec des symboles :

- ❶ < : inférieur à
- ❷ > : supérieur à
- ❸ <= : inférieur ou égal
- ❹ >= : supérieur ou égal
- ❺ == : égal à (attention, deux signes == ! Un seul = c'est pour affecter, c'est le piège classique)
- ❻ != : différent de

```
1 # Operation comparaison
2 x = 1
3 y = 3.6
4 print(x < y)    # inferieur
5 print(x > y)    # superieur
6 print(x <= y)   # inferieur ou egal
7 print(x != y)   # different
8 print(x == y)   # egalite
```

À chaque comparaison, Python vous répond True ou False. C'est comme s'il vous disait « Oui » ou « Non » sans hésiter. Il n'a pas de doute. L'ordinateur ne doute jamais. Il a tort parfois, mais il ne doute pas.

3.1.4 Opérations logiques : le ET, le OU, le XOR

Là, on rentre dans le vrai du vrai. Les opérateurs logiques, c'est la base de la réflexion conditionnelle. Ils servent à combiner plusieurs conditions.

- ❶ and (ou &) : le **ET** logique. Les deux conditions doivent être vraies.
- ❷ or (ou |) : le **OU** logique. Au moins une des deux conditions doit être vraie.
- ❸ xor (ou ^) : le **OU exclusif**. Une et une seule des deux conditions doit être vraie. C'est le « soit l'un, soit l'autre, mais pas les deux ».

```
1 # Operation de Logique
2 print(True and True)      # AND (et) -> True
3 print(False or True)     # OR (ou) -> True
4 print(False ^ True)      # XOR (ou exclusif) -> True
```

3.1.5 Combiner tout ça : la recette du succès

Vous pouvez mélanger les trois types d'opérations pour créer des expressions complexes. C'est comme quand vous dites :

« **Si** j'ai fini mes devoirs **ET** que j'ai rangé ma chambre **ALORS** je peux sortir jouer au foot. »

```
1 # Declaration des variables
2 x, y = 1, 3
3 print((x + y) >= 3 and x > 0) # True
```

Ici, Python calcule d'abord $x + y$ (ça fait 4). Puis il vérifie si 4 est supérieur ou égal à 3 (vrai). Ensuite, il vérifie si $x > 0$ (vrai). Enfin, il fait le and entre les deux. Le résultat est True. Il est content. Vous aussi.

3.2 Structures de contrôle : Conditions et Boucles

3.2.1 Le problème du jour : le videur de la boîte de nuit

Vous êtes à la sortie du lycée de Bambao. C'est vendredi soir. Vous voulez aller à la fête du quartier. Mais il y a un videur à l'entrée. Il est grand. Il est costaud. Il a un regard qui tue.

Il vous regarde. Il vous jauge. Il prend une décision.

- **Si** vous avez plus de 18 ans, il vous laisse entrer.
- **Sinon, si** vous êtes avec un grand frère qui a 18 ans, il vous laisse entrer.
- **Sinon**, il vous vire. C'est la vie.

C'est exactement comme ça que fonctionne une **condition** en programmation. L'ordinateur, c'est le videur. Il regarde une situation, il vérifie une condition, et il décide quoi faire.

3.2.2 Les conditions : le videur numérique

En Python, on utilise trois mots magiques pour les conditions : if, elif (qui veut dire "sinon si"), et else (le fourre-tout de la dernière chance).

```

1  # Exemples de conditions
2  x = 10
3  y = 5
4
5  if x > y:
6  print("x est plus grand que y")
7  elif x == y:
8  print("x est égal a y")
9  else:
10 print("x est plus petit que y")

```

Ici, Python joue le videur :

- Il regarde si x est plus grand que y. **Si** c'est vrai, il affiche "x est plus grand".
- **Sinon**, **si** x est égal à y, il affiche "x est égal".
- **Sinon** (donc si x est plus petit), il affiche "x est plus petit".

C'est binaire. C'est tranché. C'est comme le videur : il n'y a pas de « peut-être ». Tu passes ou tu passes pas.

Et vous pouvez rendre les conditions plus complexes en utilisant les opérateurs logiques qu'on a vus tout à l'heure :

```

1  age = 17
2  accompagne = True
3
4  if age >= 18 or (age >= 16 and accompagne):
5  print("Bienvenue a la fete, frere !")
6  else:
7  print("Desole, mon gars. Reviens dans deux ans.")

```

3.2.3 Les boucles : le vendeur de chips

Maintenant, imaginez que vous êtes au stade de Hamramba. Il y a un match de foot. Youssouf, le vendeur de chips, fait le tour des gradins. Il a un gros sachet de chips. Il doit le distribuer à tout le monde.

Il va parcourir chaque siège, un par un, jusqu'à ce qu'il ait fini.

C'est une **boucle**. L'ordinateur adore les boucles. Il est obsédé par la répétition. C'est son petit plaisir.

Il existe deux types de boucles :

- ❶ **La boucle for** : le facteur. Il sait exactement combien de maisons il doit visiter. Il parcourt une liste, un élément après l'autre, et quand il a fini, il s'arrête.
- ❷ **La boucle while** : le gars qui fait des pompes. Il continue tant que la condition est vraie. Si vous ne lui dites jamais d'arrêter, il continue à l'infini. Et c'est là que le drame arrive.

Boucle for : le facteur

```

1  # Boucle for : le facteur
2  fruits = ["pomme", "banane", "orange"]
3  for fruit in fruits:
4  print(fruit)

```

Ici, Python prend le premier fruit de la liste ("pomme"), il l'affiche. Puis le deuxième ("banane"), il l'affiche. Puis le troisième ("orange"), il l'affiche. Et quand il n'y a plus de fruits, il s'arrête. Il est content. Vous aussi.

Boucle while : le gars qui fait des pompes

```

1  # Boucle while : les pompes
2  i = 0
3  while i < 5:
4  print(i, "- Je fais une pompe !")
5  i += 1  # equivalent a i = i + 1

```

Ici, Python dit : « Tant que *i* est plus petit que 5, je fais une pompe. » Il commence avec *i* = 0. Il fait une pompe. Il ajoute 1 à *i*. Il recommence. Quand *i* atteint 5, il s'arrête. Il a fait 5 pompes. Il est fatigué. Mais il est fier.

Attention au piège ! Si vous oubliez la ligne *i* += 1, la condition *i* < 5 reste vraie pour toujours. La boucle tourne à l'infini. L'ordinateur chauffe. Le ventilateur s'affole. On dirait un avion qui décolle. C'est spectaculaire, mais c'est un bug. Évitez-le.

3.2.4 Combiner conditions et boucles : le match de foot

Vous pouvez mélanger les deux. Par exemple, vous voulez afficher tous les nombres de 0 à 9, mais vous voulez dire « Mwambo » pour les nombres pairs et « Pas mwambo » pour les impairs.

```

1  # Combinaison de condition et boucle
2  for i in range(10):
3  if i % 2 == 0:  # Si i est pair
4  print(f"{i} est un nombre pair. Mwambo !")
5  else:
6  print(f"{i} est un nombre impair. Pas mwambo...")

```

La boucle *for* parcourt les nombres de 0 à 9. Pour chaque nombre, elle pose la question : « Est-ce que ce nombre est pair ? » Si oui, elle affiche « Mwambo ! » Sinon, elle affiche « Pas mwambo... »

C'est beau. C'est simple. C'est de la programmation.

3.2.5 Résumons : le videur et le vendeur

- ① **Conditions** (*if*, *elif*, *else*) : le videur de la boîte de nuit. Il prend une décision en fonction d'une situation.
- ② **Boucle** *for* : le facteur. Il parcourt une liste jusqu'au bout.
- ③ **Boucle** *while* : le gars qui fait des pompes. Il continue tant qu'une condition est vraie. N'oubliez pas la condition de sortie !
- ④ **Combinaison** : la vie réelle. On prend des décisions en répétant des actions.

Maintenant, vous pouvez contrôler le flux de votre programme. Vous savez dire « Si ceci, alors cela », et « Répète ça jusqu'à ce que ».

C'est comme avoir un super-pouvoir. Vous êtes le maître du temps et des décisions. Utilisez-le avec sagesse. Et surtout, amusez-vous.

4 Structures de données : listes, tuples et dictionnaires

4.1 Le problème du jour : vous devez ranger votre vie

Vous avez des affaires partout. Des chansons, des numéros de téléphone, des coordonnées GPS, des notes de cours, des noms de potes. Vous ne savez plus où ranger quoi.

Si vous mettez tout dans une seule boîte, c'est le bazar. Si vous utilisez une boîte par affaire, vous avez trop de boîtes. Il vous faut des outils adaptés.

Python vous donne trois outils magiques :

- ① La **liste** : un tiroir à compartiments.
- ② Le **tuple** : une boîte scellée.
- ③ Le **dictionnaire** : un carnet d'adresses.

4.2 La liste : le tiroir à compartiments

C'est le plus simple. Vous ouvrez un tiroir, il y a des cases numérotées. Vous mettez un truc dans chaque case. Vous pouvez ajouter, enlever, modifier, ranger dans n'importe quel ordre.

- ❶ **Déclaration** : crochets [].
- ❷ **Accès** : `liste[0]`, `liste[1]`, etc. On commence à 0.
- ❸ **Modifiable** : on peut tout changer.

```

1 # La liste : le tiroir
2 fruits = ["pomme", "banane", "orange"]
3 print(fruits[0])      # pomme
4 fruits.append("kiwi")  # ajout
5 fruits[1] = "poire"    # modification
6 print(fruits)         # ["pomme", "poire", "orange", "kiwi"]

```

Quand utiliser une liste? Quand vous avez une collection d'éléments dans un ordre précis et que vous voulez pouvoir ajouter, supprimer ou modifier des éléments. Par exemple : une playlist, les notes de la classe, les courses du marché.

4.3 Le tuple : la boîte scellée

Imaginez une boîte en plastique dur. Vous mettez des affaires dedans, vous la fermez, et personne ne peut plus rien changer. Pas vous, pas l'ordinateur, personne. C'est gravé.

- ❶ **Déclaration** : parenthèses ().
- ❷ **Accès** : comme une liste (`tuple[0]`).
- ❸ **Non modifiable** : `tuple[0] = ?` Impossible !

```

1 # Le tuple : la boîte scellée
2 coords = (11.7033, 43.2556) # Moroni
3 print("Latitude :", coords[0])
4 # coords[0] = 12.0 # ERREUR ! Le tuple ne se modifie pas.

```

Quand utiliser un tuple? Pour des données qui ne doivent jamais changer. Par exemple : les coordonnées GPS d'un lieu, une date de naissance, les dimensions d'une image (largeur, hauteur). C'est le coffre-fort des données.

4.4 Le dictionnaire : le carnet d'adresses

Vous avez un carnet. Chaque nom est associé à un numéro de téléphone. Vous cherchez « Yousouf », vous trouvez son numéro. Vous ne parcourez pas tout le carnet, vous allez directement à la bonne page.

- ❶ **Déclaration** : accolades {}.
- ❷ **Accès** : `dictionnaire["nom"]` pour la valeur.
- ❸ **Modifiable** : on peut ajouter, modifier, supprimer des entrées.


```

1  # Le dictionnaire : le carnet d'adresses
2  contacts = {
3      "Youssef": "777 12 34",
4      "Soirfane": "777 56 78"
5  }
6
7  print("Numero de Youssef :", contacts["Youssef"])  # 777 12 34
8
9  # Ajouter un contact
10 contacts["Hadidja"] = "777 90 12"
11
12 # Modifier un contact
13 contacts["Youssef"] = "777 99 99"
14
15 # Supprimer un contact
16 del contacts["Soirfane"]
17
18 print(contacts)  # {"Youssef": "777 99 99", "Hadidja": "777 90 12"}

```

Quand utiliser un dictionnaire? Quand vous voulez associer des noms (ou des identifiants) à des valeurs. Par exemple : un annuaire, un catalogue de produits (nom → prix), le classement d'une classe (élève → note).

4.5 Tableau récapitulatif : lequel choisir ?

	Liste	Tuple	Dictionnaire
Syntaxe	[]	()	{}
Accessible par	indice (0,1,2...)	indice (0,1,2...)	clé (nom, identifiant)
Modifiable?	Oui	Non	Oui
Ordre?	Oui	Oui	Non (depuis Python 3.7, oui)
Quand l'utiliser?	Séquence ordonnée modifiable	Données fixes	Association clé → valeur

4.6 Application pratique : l'annuaire du lycée de Bambao

On va créer un petit système qui gère les élèves du lycée. On va utiliser les trois structures ensemble.

```

1  # Une liste d'eleves (ordre alphabetique)
2  eleves = ["Youssef", "Soirfane", "Hadidja", "Ali"]
3
4  # Un tuple pour les coordonnees du lycee (fixes)
5  lycee_coords = (11.7033, 43.2556)
6
7  # Un dictionnaire pour les classes et ages
8  annuaire = {
9      "Youssef": {"classe": "Terminale", "age": 17},
10     "Soirfane": {"classe": "Premiere", "age": 16},
11     "Hadidja": {"classe": "Terminale", "age": 18},
12     "Ali": {"classe": "Seconde", "age": 15}
13 }
14
15 # Afficher tout le monde
16 print("Liste des eleves :", eleves)
17 print("Coordonnees du lycee :", lycee_coords)
18
19 for nom in eleves:
20     infos = annuaire[nom]

```

```
21 print(nom, "est en", infos["classe"], "et a", infos["age"], "ans.")
```

4.7 Conclusion : vous êtes un organisateur de génie

Avec ces trois outils, vous pouvez ranger n'importe quoi :

- ① La **liste** pour les collections ordonnées et modifiables.
- ② Le **tuple** pour les données fixes qui ne doivent jamais changer.
- ③ Le **dictionnaire** pour associer facilement un nom à une valeur.

Maintenant, quand on vous dit « range tes données », vous répondez « liste, tuple ou dictionnaire ? » et vous choisissez l'outil parfait. C'est ça, être un vrai programmeur. Allez, on passe à la suite !

5 Fonctions : votre petit robot personnel

5.1 Le problème du jour : le vendeur de beignets

Youssef tient un petit stand de beignets au marché de Volo-Volo. Chaque jour, il doit calculer le prix total de ses ventes. Il a 3 types de beignets :

- Les beignets nature : 300 francs pièce.
- Les beignets fourrés au chocolat : 500 francs pièce.
- Les beignets fourrés à la confiture : 450 francs pièce.

Vous vous dites : « Je vais écrire un programme. »

Vous commencez à écrire :

```
1 # Calcul pour un client qui achete 2 nature, 3 chocolat, 1 confiture
2 total = 2 * 300 + 3 * 500 + 1 * 450
3 print("Total :", total)
```

Puis un autre client arrive. Vous écrivez encore :

```
1 # Un autre client : 5 nature, 2 chocolat, 4 confiture
2 total = 5 * 300 + 2 * 500 + 4 * 450
3 print("Total :", total)
```

Puis un autre. Et un autre. Et un autre.

Au bout du 50ème client, vous êtes fatigué. Vous copiez-collez la même ligne encore et encore, en changeant juste les nombres. C'est long. C'est chiant. C'est inutile.

Si vous étiez un vrai programmeur, vous vous diriez : « Je vais écrire cette formule une fois, et je vais l'appeler quand j'en ai besoin. »

C'est exactement ce que font les **fonctions**.

5.2 Une fonction, c'est une recette de cuisine

Vous mettez des ingrédients (ce qu'on appelle des **paramètres**), vous suivez les étapes (les instructions), et ça vous sort un plat (la **valeur retournée**). L'avantage, c'est que vous pouvez réutiliser la recette autant de fois que vous voulez sans avoir à réécrire les étapes.

```

1  # Definition d'une fonction qui calcule le total pour Youssouf
2  def prix_beignets(nature, chocolat, confiture):
3      total = nature * 300 + chocolat * 500 + confiture * 450
4      return total
5
6  # Utilisation de la fonction pour différents clients
7  print("Client 1 :", prix_beignets(2, 3, 1))  # 2 nature, 3 chocolat, 1
      confiture
8  print("Client 2 :", prix_beignets(5, 2, 4))  # 5 nature, 2 chocolat, 4
      confiture
9  print("Client 3 :", prix_beignets(0, 10, 0)) # 10 chocolat

```

Ce que vous devez retenir :

- ① Une fonction se définit avec le mot-clé `def`, suivi du nom, de parenthèses contenant les paramètres, et d'un deux-points.
- ② Le bloc d'instructions est indenté (comme pour les conditions et les boucles).
- ③ Le mot-clé `return` renvoie le résultat. Si vous ne mettez pas de `return`, la fonction renvoie `None` (rien).

5.3 Pourquoi utiliser des fonctions ?

Parce que vous êtes fainéant, et c'est une bonne chose !

- ❶ **Réutilisabilité** : vous écrivez le code une fois, vous l'utilisez mille fois.
- ❷ **Clarté** : votre programme devient plus facile à lire. `prix_beignets(2, 3, 1)`, on comprend tout de suite ce que ça fait.
- ❸ **Correction facile** : si vous vous trompez dans la formule (par exemple, le prix des beignets change), vous modifiez un seul endroit. Une seule ligne. C'est magique.

5.4 Les fonctions, c'est comme des robots

Imaginez que vous avez un petit robot dans votre cuisine. Vous lui dites : « Robot, fais-moi un café. » Il le fait. Vous ne lui redemandez pas de vous expliquer comment il le fait. Il le fait.

Une fonction, c'est pareil. Vous lui donnez un nom (`prix_beignets`), vous lui donnez des ingrédients (les paramètres), et il vous rend un résultat. C'est un robot.

5.5 Les fonctions sans paramètres

Parfois, vous n'avez pas besoin d'ingrédients. La fonction fait juste son boulot sans rien attendre de vous.

```

1  # Une fonction qui dit bonjour
2  def dire_bonjour():
3      print("Salut la team !")
4      print("Bienvenue dans le cours de Python.")
5
6  # Appel de la fonction
7  dire_bonjour()

```

5.6 Les fonctions avec des valeurs par défaut

Parfois, vous voulez que votre robot ait un comportement par défaut, mais que vous puissiez le changer si vous voulez.

```
1 # Fonction avec valeur par défaut pour le chocolat
2 def prix_beignets(nature, chocolat=0, confiture=0):
3     total = nature * 300 + chocolat * 500 + confiture * 450
4     return total
5
6 # Appels
7 print("Sans chocolat :", prix_beignets(4, 0, 2))    # 4 nature, 2 confiture
8 print("Avec chocolat :", prix_beignets(2, 3, 1))    # 2 nature, 3 chocolat, 1
9             confiture
```

5.7 Les fonctions qui retournent plusieurs valeurs

Votre robot peut même vous rendre plusieurs choses à la fois. Par exemple, le total et le nombre d'articles.

```
1 # Fonction qui retourne le total et le nombre d'articles
2 def total_et_quantite(nature, chocolat, confiture):
3     total = nature * 300 + chocolat * 500 + confiture * 450
4     quantite = nature + chocolat + confiture
5     return total, quantite
6
7 # Utilisation
8 resultat = total_et_quantite(2, 3, 1)
9 print("Total :", resultat[0], "Quantite :", resultat[1])
10
11 # Ou plus simple :
12 total, quantite = total_et_quantite(2, 3, 1)
13 print("Total :", total, "Quantite :", quantite)
```

5.8 Application pratique : le robot du lycée

On va créer un petit robot qui vous aide pour vos devoirs.

```
1 # Le robot du lycee
2 def moyenne(notes):
3     return sum(notes) / len(notes)
4
5 def appreciation(moyenne):
6     if moyenne >= 16:
7         return "Excellent ! Tu es un genie."
8     elif moyenne >= 12:
9         return "Bien. Continue comme ca."
10    elif moyenne >= 10:
11        return "Passable. Tu peux mieux faire."
12    else:
13        return "Il va falloir bosser, mon gars."
14
15 # Utilisation
16 mes_notes = [14, 12, 16, 9, 18]
17 moy = moyenne(mes_notes)
18 app = appreciation(moy)
19
20 print("Tes notes :", mes_notes)
21 print("Moyenne :", moy)
```

22 `print("Appreciation :", app)`

5.9 Conclusion : vous avez des robots partout

Avec les fonctions, vous pouvez :

- ① Écrire du code une fois, l'utiliser partout.
- ② Rendre votre code plus clair et plus propre.
- ③ Corriger facilement vos erreurs.
- ④ Créer vos propres robots pour faire n'importe quoi.

C'est le super-pouvoir ultime. Vous n'êtes plus un simple exécutant, vous êtes un chef d'orchestre. Vous organisez. Vous structurez. Vous déléguez.

La prochaine fois que vous écrirez un programme, demandez-vous : « Est-ce que je peux mettre ça dans une fonction ? » Et si la réponse est oui, faites-le. Votre futur vous remerciera.

6 Les bibliothèques : le copinage

6.1 Le problème du jour : le concours de pronostics de Youssef

Youssef est un grand fan de foot. Chaque week-end, il regarde les matchs de la coupe des Comores. Il organise un concours de pronostics avec ses potes du lycée de Bambao.

Il leur demande : « Est-ce que vous croyez que l'équipe de Ngazidja va gagner contre l'équipe de Mwali ? »

Mais Youssef n'est pas juste un fan, il est aussi un programmeur. Il veut automatiser son concours. Il veut que l'ordinateur lance un dé pour décider du vainqueur.

Il se dit : « Je vais écrire une fonction qui lance un dé. »

Mais comment faire pour avoir un nombre aléatoire en Python ?

Python, tout seul dans son coin, ne sait pas faire de l'aléatoire. Il est trop sérieux. Il est trop rationnel. Il a besoin d'aide.

C'est là qu'interviennent les **bibliothèques**.

6.2 Importer une bibliothèque : l'entrepôt à outils

Python est un langage riche, mais il n'a pas tout dans ses poches. Heureusement, des milliers de développeurs ont écrit des fonctions toutes faites, organisées en **bibliothèques** (aussi appelées *modules*). Vous pouvez les importer chez vous, comme si vous empruntiez des livres dans une bibliothèque géante.

Pour utiliser une bibliothèque, on écrit `import nom_du_module`. Une fois importée, on peut utiliser toutes ses fonctions en les préfixant par le nom du module.

```
1 import random # J'ouvre la porte de l'entrepot "random"
2
3 # Je prends une fonction qui me donne un nombre aleatoire entre 1 et 6
4 de = random.randint(1, 6)
5 print("Le de est tombe sur :", de)
```

Ce que vous devez retenir :

- ① `import` se met en haut du fichier.

- ② Pour appeler une fonction d'un module, on écrit `module.fonction()`.
- ③ Il existe des modules pour tout : mathématiques, gestion de fichiers, création de jeux, et bien sûr, le Deep Learning !

6.3 La fonction "lancer de dé" version fonction + module

Maintenant, on peut créer notre propre fonction qui utilise la magie de `random`.

```
1 import random
2
3 def lancer_de():
4     return random.randint(1, 6)
5
6 # Test
7 print("Lancer 1 :", lancer_de())
8 print("Lancer 2 :", lancer_de())
9 print("Lancer 3 :", lancer_de())
```

Vous venez de combiner trois concepts : fonction, module, et boucle. Vous êtes en train de grimper dans la hiérarchie du code. Respectez-vous.

6.4 Le module `random` : le roi du hasard

Le module `random` est un petit génie. Il sait faire plein de choses avec le hasard.

```
1 import random
2
3 # Lancer un de
4 print(random.randint(1, 6))
5
6 # Choisir un fruit au hasard dans une liste
7 fruits = ["pomme", "banane", "orange", "kiwi"]
8 fruit_aleatoire = random.choice(fruits)
9 print("Aujourd'hui, je mange une", fruit_aleatoire)
10
11 # Melanger une liste
12 cartes = ["as", "roi", "dame", "valet", "10"]
13 random.shuffle(cartes)
14 print("Cartes melangees :", cartes)
```

6.5 Le module `math` : la calculatrice surpuissante

Si vous avez besoin de faire des calculs avancés, il y a `math`.

```
1 import math
2
3 # Racine carree
4 print("Racine de 16 :", math.sqrt(16)) # 4.0
5
6 # Valeurs constantes
7 print("Pi :", math.pi) # 3.141592653589793
8 print("Exponentiel :", math.e) # 2.718281828459045
9
10 # Arrondir a l'entier superieur
11 print("3.2 arrondi au superieur :", math.ceil(3.2)) # 4
```

6.6 Importer en donnant un surnom

Parfois, le nom du module est trop long à écrire. On peut lui donner un surnom avec `as`.

```
1 import random as rd
2 import math as mt
3
4 print(rd.randint(1, 6)) # random devient rd
5 print(mt.sqrt(25))      # math devient mt
```

6.7 Importer seulement une fonction spécifique

Si vous n'avez besoin que d'une seule fonction d'un module, vous pouvez l'importer directement.

```
1 from random import randint
2
3 print(randint(1, 6)) # Pas besoin de écrire random.
```

Attention : cette méthode est pratique, mais si vous importez plusieurs fonctions de plusieurs modules, vous pouvez vous perdre. Utilisez-la avec parcimonie.

6.8 Application pratique : le concours de pronostics de Youssef

On va créer un petit programme qui choisit le vainqueur d'un match de foot au hasard.

```
1 import random
2
3 def match_aleatoire(equipe1, equipe2):
4     gagnant = random.choice([equipe1, equipe2])
5     return gagnant
6
7 # Programme principal
8 equipeA = "Ngazidja"
9 equipeB = "Mwali"
10
11 for i in range(5):
12     gagnant = match_aleatoire(equipeA, equipeB)
13     print("Match", i+1, ":", gagnant, "gagne !")
```

6.9 Conclusion : les bibliothèques, votre armée secrète

Les bibliothèques, c'est votre armée de robots invisibles qui font le boulot à votre place. Avec elles, vous pouvez :

- ① Générer du hasard avec `random`.
- ② Faire des maths complexes avec `math`.
- ③ Lire et écrire des fichiers avec `os` et `sys`.
- ④ Créer des jeux avec `pygame`.
- ⑤ Et surtout, faire du **Deep Learning** avec `tensorflow` ou `torch` !

La semaine prochaine, on va utiliser tout ce qu'on a appris pour faire quelque chose de vraiment magique : on va entraîner un ordinateur à reconnaître des images.

Alors, préparez-vous. Ça va être spectaculaire.

"Un programmeur paresseux est un bon programmeur. Il trouve les raccourcis pour ne pas travailler. Et ces raccourcis, ce sont les bibliothèques."

7 Projets

7.1 Le problème du jour : vous êtes un agent secret

Bravo, agent secret. Vous avez terminé votre formation de base. Vous savez maintenant :

- ① Ranger des informations dans des variables.
- ② Faire des calculs comme un calculateur humain.
- ③ Prendre des décisions avec les conditions.
- ④ Répéter des actions avec les boucles.
- ⑤ Organiser vos données avec des listes, des tuples et des dictionnaires.
- ⑥ Créer vos propres robots avec des fonctions.
- ⑦ Utiliser les bibliothèques des autres pour gagner du temps.

Vous êtes prêt pour votre première mission.

Votre mission, si vous l'acceptez, consiste à réaliser trois projets. Vous pouvez les faire dans l'ordre ou pêle-mêle, comme vous voulez. L'important est de tester, de planter, de recommencer, et de triompher.

Projet 1 : La calculette de l'apocalypse

Youssef tient son stand de beignets au marché de Volo-Volo. Il est fatigué de faire les calculs à la main. Il veut une calculette qui fait tout. Il veut additionner, soustraire, multiplier, diviser, et même calculer des puissances.

Écrivez un programme qui demande deux nombres à l'utilisateur, puis affiche :

- ❶ Leur somme (+).
- ❷ Leur différence (-).
- ❸ Leur produit (*).
- ❹ Leur division réelle (/).
- ❺ Leur division entière (/).
- ❻ Le premier nombre élevé à la puissance du second (**).

Astuce :

Utilisez `input()` pour demander les nombres, et convertissez-les en `float()` ou `int()` selon vos besoins. N'oubliez pas que `input()` renvoie toujours du texte (une `str`). Vous devez le transformer en nombre avant de faire des calculs.

Exemple de squelette :

```
1 # La calculette de Youssef
2 print("Bienvenue dans la calculette de l'apocalypse !")
3
4 # Demander les deux nombres
5 nombre1 = float(input("Donne le premier nombre : "))
6 nombre2 = float(input("Donne le deuxieme nombre : "))
7
```



```

8  # Faire les calculs
9  print("Somme :", nombre1 + nombre2)
10 print("Difference :", nombre1 - nombre2)
11 print("Produit :", nombre1 * nombre2)
12 print("Division reelle :", nombre1 / nombre2)
13 print("Division entiere :", nombre1 // nombre2)
14 print("Puissance :", nombre1 ** nombre2)

```

Projet 2 : Le détecteur de pair/impair

Soirfane est un grand fan de foot. Il veut organiser des équipes pour le match du vendredi. Il veut savoir si le nombre total de joueurs est pair ou impair.

Écrivez un programme qui demande un nombre à l'utilisateur, puis affiche s'il est pair ou impair.

Bonus :

Affichez tous les nombres pairs de 0 jusqu'au nombre saisi.

Exemple de squelette :

```

1  # Le detecteur de pair/impair de Soirfane
2  print("Le detecteur de pair/impair est la !")
3
4  nombre = int(input("Donne un nombre entier : "))
5
6  if nombre % 2 == 0:
7      print(nombre, "est un nombre pair.")
8  else:
9      print(nombre, "est un nombre impair.")
10
11 # BONUS : afficher tous les nombres pairs de 0 a nombre
12 print("Les nombres pairs de 0 a", nombre, "sont :")
13 for i in range(0, nombre + 1, 2):
14     print(i, end=" ")
15 print() # Pour un retour a la ligne

```

Projet 3 : La machine à insulter (version améliorée)

Hadidja est la plus gentille du lycée de Bambao. Mais elle a un secret : elle a créé un robot qui insulte ses potes... gentiment. Elle veut une fonction qui prend un nom en paramètre et qui renvoie une phrase du type :

« Hé [nom], tu es une [insulte aléatoire] ! »

Instructions :

- ❶ Créez une liste de 5 insultes amicales. Par exemple : « tête de patate », « lapin crétin », « narvalo », « koala drogué », « génie du clavier ».
- ❷ Écrivez une fonction `machine_a_insulter(nom)` qui :
 - Prend un nom en paramètre.
 - Choisit une insulte au hasard dans la liste (utilisez le module `random`).
 - Renvoie la phrase formatée.
- ❸ Appelez cette fonction trois fois pour trois noms différents (vous pouvez les demander à l'utilisateur avec `input()`).

Exemple de squelette :

```

1  # La machine a insulter de Hadidja
2  import random
3
4  # 1. La liste des insultes
5  insultes = ["tete de patate", "lapin cretin", "narvalo", "koala drogue", "genie
6             du clavier"]
7
8  # 2. La fonction
9  def machine_a_insulter(nom):
10     insulte = random.choice(insultes)
11     return "He " + nom + ", tu es un(e) " + insulte + " !"
12
13 # 3. Test avec trois utilisateurs
14 nom1 = input("Nom du premier victime : ")
15 nom2 = input("Nom du deuxieme victime : ")
16 nom3 = input("Nom du troisieme victime : ")
17
18 print(machine_a_insulter(nom1))
19 print(machine_a_insulter(nom2))
20 print(machine_a_insulter(nom3))

```

7.2 Conseil du samedi soir

Vous allez planter. C'est normal. Ce n'est pas grave. Tout le monde plante. Les programmeurs professionnels plantent tous les jours. La différence, c'est qu'ils savent lire les messages d'erreur.

Quand vous voyez une erreur rouge :

- ① **Lisez-la** : elle vous dit où (le numéro de ligne) et quoi (le type d'erreur).
- ② **Respirez** : ce n'est pas la fin du monde.
- ③ **Corrigez** : une petite faute à la fois.
- ④ **Relancez** : le code pour voir si ça marche.
- ⑤ **Répétez** : jusqu'à ce que ça marche.

7.3 Conclusion : vous êtes des héros

En une semaine, vous avez appris ce qu'un lycéen normal met un semestre à comprendre. Vous êtes maintenant capables de :

- ① Lire et écrire du code Python.
- ② Comprendre ce qu'est une variable, une condition, une boucle, une liste, une fonction, une bibliothèque.
- ③ Résoudre des problèmes simples avec des programmes.
- ④ Et surtout, ne plus avoir peur de l'ordinateur.

La semaine prochaine, on va utiliser tout ça pour faire quelque chose de vraiment spectaculaire : on va entraîner un ordinateur à reconnaître des images. On va utiliser des bibliothèques de Deep Learning. On va cliquer sur « Jouer », et la magie va opérer.

Préparez-vous. Ça va être épique.

"L'ordinateur est un outil. La programmation est un super-pouvoir. Et vous, vous venez d'apprendre à l'utiliser."